National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

| | |
|---|---|
| **Lab Number:** | 06 |
| **Course Code:** | CL2001 |
| **Course Title:** | Data Structures - Lab |
| **Semester:** | Fall 2026 |
| **Instructor:** | Muhammad Mehdi |
| **Email:** | muhammad.mehdi@nu.edu.pk |

# Guidelines

- You may use **only the concepts covered in the lab** to complete this task.
- Your submission must be a **single PDF file** that includes:
  - The source code (in plain text)
  - Screenshots of the corresponding outputs
- Name your file using the following format: **rollno_name_labtasknumber.pdf**
  **Example:** *25p-1234_ozair_labtask06.pdf*
- Unethical use of AI tools will result in **0 marks**.
- Late submissions will incur a penalty of **-3 marks per day**.

# Tasks

**ONLY USE POINTER NOTATION TO IMPLEMENT ALL THE SOLUTIONS. ARRAY INDEXING [ ] IS NOT ALLOWED.**

## 1 Palindrome Linked List

Given a linked list as a parameter, define a **non-member function** that determines whether the linked list is a palindrome. The function should return `true` if the elements of the list read the same from both ends and `false` otherwise.

**Example 1:**

```
Input: list = [1,2,2,1]
Output: true
```

**Example 2:**

```
Input: list = [1,2]
Output: false
```

## 2 Valid Parentheses

Given a string `s` containing only the characters `'('`, `')'`, `'{'`, `'}'`, `'['`, and `']'`, write a **procedural function** that determines whether the string contains valid parentheses.

A string is valid if:

- Every opening bracket is closed by a bracket of the same type.
- Brackets are closed in the correct order.
- Every closing bracket has a corresponding opening bracket.

**Example 1:**

```
Input: s = "()"
Output: true
```

**Example 2:**

```
Input: s = "()[]{}"
Output: true
```

**Example 3:**

```
Input: s = "(]"
Output: false
```

**Example 4:**

```
Input: s = "([])"
Output: true
```

**Example 5:**

```
Input: s = "([)]"
Output: false
```

# 3 Baseball Game

You are keeping the scores for a baseball game with a set of special rules. At the beginning of the game, the record is empty.

Given a list of strings `operations`, where each operation represents an action on the score record, process all operations according to the following rules:

- An integer `x`: Record a new score of `x`.
- `'+'`: Record a new score equal to the sum of the previous two scores.
- `'D'`: Record a new score equal to double the previous score.
- `'C'`: Invalidate and remove the previous score.

Return the sum of all scores remaining in the record after applying all operations.

**Example 1:**

```
Input: ops = ["5","2","C","D","+"]
Output: 30
```

Explanation:

- "5": Add 5, record is [5].
- "2": Add 2, record is [5, 2].
- "C": Remove the previous score, record is [5].
- "D": Add 2 × 5 = 10, record is [5, 10].
- "+": Add 5 + 10 = 15, record is [5, 10, 15].

The total sum is 5 + 10 + 15 = 30.

**Example 2:**

```
Input: ops = ["5","-2","4","C","D","9","+","+"]
Output: 27
```

**Example 3:**

```
Input: ops = ["1","C"]
Output: 0
```

Explanation:

- "1": Add 1, record is [1].
- "C": Remove the previous score, record is [ ].

Since the record is empty, the total sum is 0.

# 4 Undo and Redo

Write a program that implements and demonstrates **undo and redo functionality** using a stack-based approach.

The program should simulate actions in a simple car-control system. Repeatedly display a menu allowing the user to perform actions such as:

- Accelerate
- Brake
- Decelerate
- Turn Left

- Turn Right
- Quit

Each action performed by the user should be recorded so that it can later be undone.

The program should also provide the following controls:

- Press Z to **undo** the most recent action.
- Press Y to **redo** the most recently undone action.
- Press Q to **quit** the program.

The program should maintain the appropriate history so that multiple consecutive undo and redo operations are supported. When a new action is performed after an undo, the redo history should be cleared.

**Example Program:**

```
Car Control Menu
1. Accelerate
2. Brake
3. Decelerate
4. Turn Left
5. Turn Right
Z. Undo
Y. Redo
Q. Quit

Choice: 1
Action: Accelerate

Choice: 5
Action: Turn Right

Choice: Z
Undo: Turn Right

Choice: Z
Undo: Accelerate

Choice: Y
Redo: Accelerate
```